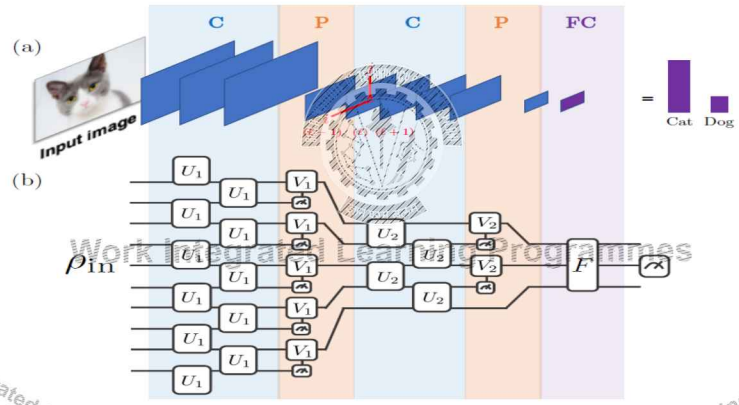
QCNN Structure



Pooling via Mid-Circuit Measurement

QCNN Pooling step by step:

1. **Measure** qubit q_k in the computational basis. Result: classical bit $m_k \in \{0, 1\}$. q_k is now collapsed - removed from the active circuit.
2. **Feed forward** m_k classically to the next gate.
3. **Apply correction** unitary $V_j(m_k)$ to the surviving neighbouring qubit q_j. This is a classically-controlled quantum operation - it injects information from the measured qubit into the survivor.

What is achieved?	Why it matters
Dimension reduction	q_k is gone; active qubit count halves - the same purpose as max-pool
Information transfer	m_k carries partial information about q_k's state; $V_j(m_k)$ injects it into the surviving qubit - unlike max-pool which simply discards

Parameter Efficiency: Cong's QCNN

Cong's QCNN parameter count is logarithmic:
Each pooling layer halves the active qubits, so only $\log_2 N$ convolution-pooling stages are required. One shared unitary $U(\theta)$ is learned at each stage.

N (qubits)	QCNN rounds ($\log_2 N$)	HEA gate groups ($N/2 \times L$, $L = 3$)	Ratio
8	3	12	4×
16	4	24	6×
64	6	96	16×
1024	10	1536	154×

QCNN can handle large quantum systems where HEA would face severe barren plateau risk.

No Barren Plateaus: Why QCNN Trains

Pesah et al. (2021): For QCNNs with local cost functions (measuring 1–2 final qubits), gradient variance decays *polynomially* with N, not exponentially like HEA:

$$\text{Var}\left[\frac{\partial \mathcal{L}}{\partial \theta}\right] = \Omega\left(\frac{1}{\text{poly}(N)}\right)$$

Why? Three structural reasons:

1. Each convolutional gate $U(\theta)$ acts on 2 neighbouring qubits. Its gradient depends only on the *reduced state of those 2 qubits*, not on the full 2^N-dimensional Hilbert space.
2. The cost function sees the final qubit's state.
3. Mid-circuit measurement breaks global mixing.

A 1024-qubit QCNN is trainable. A 1024-qubit HEA with global cost is not.

QCNN Applications

QCNN takes a *quantum state* as input Applications

- Quantum Phase Recognition (QPR)
- Quantum Error Correction (QEC)
- Quantum Chemistry

Connection to classical ML:

- QPR ↔ image classification
- QEC optimisation ↔ compression / decompression
- Quantum state classification ↔ binary classification

Note: for classical data (images, tabular), the HEA hybrid pipeline is the standard approach. QCNN adds value specifically when the input is already quantum.

QCNN vs. HEA: Design Choice

Property	HEA VQC	QCNN (Cong et al.)
Input type	Classical (after CNN encoder)	Quantum states ρ_{in}
Architecture	Flat; all qubits active always	Hierarchical; halves each round
Weight sharing	None (independent per qubit)	Yes (same $U(\theta)$ per round)
Param groups	$O(nL)$	$O(\log N)$
Barren plateaus	Risk with global cost	Provably free (local cost)
Nonlinearity	Final measurement only	At each pooling step
Best for	Hybrid ML on classical data	Phase recognition, QEC

Design Choice

Discussion question:

Scenario A: Input = 28×28 MNIST image.
Classify digit 0 vs 1.
Which architecture?

Scenario B: Input = 64-qubit quantum state from a quantum simulator.
Classify phase of matter.
Which architecture?

Design Choice

Discussion question:

Scenario A: Input = 28×28 MNIST image.
Classify digit 0 vs 1.
Which architecture?

Answer A: HEA hybrid.
Classical CNN encoder compresses $784 \to n$ features; HEA classifies.
QCNN not applicable - input is classical.

Scenario B: Input = 64-qubit quantum state from a quantum simulator.
Classify phase of matter.
Which architecture?

Answer B: QCNN.
Input is already quantum; QCNN handles 64 qubits with only 6 rounds; HEA would face barren plateaus.

QCNN vs. Quanvolutional Neural Networks

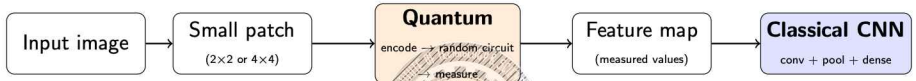
Same inspiration (classical CNNs), different architectures.

	QCNN	Quanvolution
Scope	Entire network is quantum	Only the first layer is quantum
Structure	Hierarchical - convolution + pooling rounds	A single preprocessing layer
Circuit	Trainable, problem-specific	Usually fixed or random, untrained
Qubits	Reduced progressively via pooling	Small, fixed - one patch at a time
What follows	Nothing - the circuit's output is the answer	An ordinary classical CNN

Distinction

QCNN → the whole model is quantum. Quanvolution → only feature extraction is quantum; everything after that is a classical CNN.

How Quanvolution Works



Every patch is processed independently by the same small quantum circuit - instead of a classical convolution kernel sliding over the image, a quantum circuit does. Its measurement outcomes become the new pixel values. The resulting transformed image is then handed to an entirely classical CNN.

The quantum computer here is only a feature extractor - a fancy, fixed image filter. It does not learn, and it does not classify.

Table of Contents

Why Combinatorial Optimisation Is Hard

The basic problem: find the best assignment of n binary variables $z = (z_1, \ldots, z_n) \in \{0,1\}^n$ to maximise a cost function $C(z)$.

Why brute force fails:

- Search space: 2^n strings. At $n = 50$: more than 10^{15} candidates.
- 2^n evaluations is exponential.
- No known classical algorithm solves it in polynomial time (**NP-hard**).

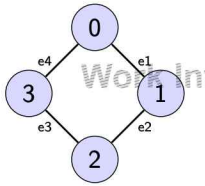
Real problems in this class:

Problem	What z and $C(z)$ mean
Network partitioning	$z_i = 0/1$ assigns a server; C = minimise inter-partition traffic
Portfolio selection	$z_i = 1$ means "buy asset i"; C = maximise return
Logistics scheduling	z_i assigns a delivery to a vehicle; C = minimise total distance

MaxCut: The Problem QAOA Was Designed For

Given: an undirected graph $G = (V, E)$ with n vertices and m edges.
Goal: partition vertices into two sets S and $\bar{S}$ to maximise the number of edges that cross the partition (cut edges).

Example - 4-node cycle:



Variables: $z_i \in \{0, 1\}$; $z_i = 0 \Rightarrow$ node $i \in S$, $z_i = 1 \Rightarrow$ node $i \in \bar{S}$.

Cost per edge (j, k):

$$C_{jk}(z) = \begin{cases} 1 & z_j \neq z_k \text{ (edge cut)} \\ 0 & z_j = z_k \end{cases}$$

Total cost: $C(z) = \sum_{(j,k)\in E} C_{jk}(z)$

Enumerate All Solutions (4-Node Cycle)

For our 4-node cycle, we can enumerate all $2^4 = 16$ strings:

String z	$C(z)$	String z	$C(z)$	String z	$C(z)$
0000	0	0101	**4**	1010	**4**
0001	2	0110	2	1011	2
0010	2	0111	2	1100	2
0011	2	1000	2	1101	2
0100	2	1001	2	1110	2
				1111	0

What QAOA must do: start from a uniform superposition over all 16 strings, then apply quantum operations that boost the amplitude of $|0101\rangle$ and $|1010\rangle$ while suppressing the others.

Step 1: Encode it into a Quantum Hamiltonian

The key idea: replace classical bits $z_j \in \{0, 1\}$ with quantum operators so the cost function becomes a Hamiltonian which helps to search for the optimal solution.

The substitution rule: $z_j \longrightarrow \frac{I - Z_j}{2}$

Because $Z_j |0\rangle = +|0\rangle$ and $Z_j |1\rangle = -|1\rangle$, the operator $\frac{I-Z_j}{2}$ acts as 0 on $|0\rangle$ and 1 on $|1\rangle$ - exactly like the classical bit z_j.

Cost per edge becomes: $C_{jk}(z) = \frac{1 - s_j s_k}{2} \longrightarrow \frac{1 - Z_j Z_k}{2}$

Problem Hamiltonian for MaxCut:

$$H_P = \sum_{(j,k)\in E} \frac{I - Z_j Z_k}{2}$$

Why this is the right object: computational basis states $|z\rangle$ are eigenstates of every Z_j simultaneously, so $H_P |z\rangle = C(z) |z\rangle$. **Measuring H_P on $|z\rangle$ returns exactly the classical cost.**

For our 4-node cycle:

$$H_P |0101\rangle = 4 |0101\rangle, \quad H_P |0000\rangle = 0 |0000\rangle$$

Confirmed: eigenvalue = cut value

The Problem Hamiltonian: Full Eigenvalue Table

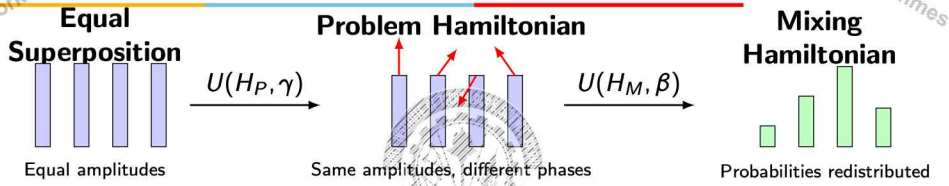
Full Eigen value table for H_P

$\|z\rangle$	$H_P \|z\rangle$	$\|z\rangle$	$H_P \|z\rangle$	$\|z\rangle$	$H_P \|z\rangle$
$\|0000\rangle$	0	$\|0101\rangle$	**4**	$\|1010\rangle$	**4**
$\|0001\rangle$	2	$\|0110\rangle$	2	$\|1011\rangle$	2
$\|0010\rangle$	2	$\|0111\rangle$	2	$\|1100\rangle$	2
$\|0011\rangle$	2	$\|1000\rangle$	2	$\|1101\rangle$	2
$\|0100\rangle$	2	$\|1001\rangle$	2	$\|1110\rangle$	2
				$\|1111\rangle$	0

How can a quantum circuit use these eigenvalues to search for the optimal solution? In QAOA, this information is encoded as **quantum phases** by the unitary

$$U(H_P, \gamma) = e^{-i\gamma H_P}.$$

How the Problem Hamiltonian Works



- $U(H_P, \gamma)$ leaves the amplitudes unchanged; adds a phase based on $C(z)$.
- Better/poorer solutions acquire different phases; their measurement probabilities are unchanged.
- $U(H_M, \beta)$ mixes the basis states, converting these phase differences into constructive and destructive interference.

Step 2: The Mixing Hamiltonian

The Mixing Hamiltonian: converts the phase differences into different measurement probabilities.

$$H_M = \sum_{j=0}^{n-1} X_j$$

where X_j is the Pauli-X (bit-flip) operator acting on qubit j.

Why choose the Pauli-X operator?
The Pauli-X operator flips a qubit: $\sigma^x|0\rangle = |1\rangle$, $\sigma^x|1\rangle = |0\rangle$.

Its unitary evolution, $e^{-ij} = R_X(2\beta)$, creates a superposition of the original and flipped states, enabling constructive and destructive interference.

The parameter β controls how strongly neighbouring solutions exchange probability amplitudes. **Small** β performs a local search, **Large** β enables broader exploration.

Putting Everything Together

Step 1 - Initialise Prepare the equal superposition

$$|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_{z\in\{0,1\}^n} |z\rangle .$$

Step 2 - Problem Hamiltonian $U(H_P, \gamma) = e^{-i\gamma H_P}$.

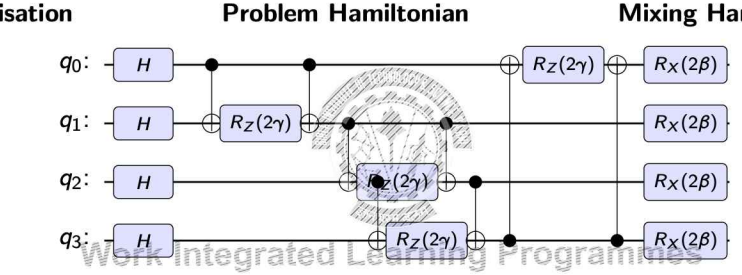
Step 3 - Mixing Hamiltonian $U(H_M, \beta) = e^{-i\beta H_M}$.

Step 4 - Measure Obtain one candidate solution over multiple shots $F_1(\gamma, \beta) = \langle H_P \rangle$. A classical optimizer (e.g., COBYLA or SPSA) updates (γ, β).

One QAOA layer = Phase Encoding + Mixing

For depth p, repeat this pair of operations p times, requiring only $2p$ **trainable parameters** $(\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p)$.

Implementing One QAOA Layer ($p = 1$)



- Every graph edge contributes one ZZ interaction, implemented using **CNOT** - $R_Z(2\gamma)$ - **CNOT**.
- The Mixing Hamiltonian is much simpler: one $R_X(2\beta)$ rotation per qubit.

Case Study: State Trace at $\gamma = \beta = \pi/8$

State after Step 1 - Hadamards (uniform superposition):

$$|s\rangle = \frac{1}{4} \sum_{z \in \{0,1\}^4} |z\rangle$$

All 16 basis states have equal amplitude 0.25 and zero phase.

State after Step 2 - $U(H_P, \gamma = \pi/8)$: Each $|z\rangle$ receives a phase $e^{-i(\pi/8)\,C(z)}$:

Cost $C(z)$	States	Phase factor	Phase (degrees)
$C = 0$	$\|0000\rangle$, $\|1111\rangle$	$e^0 = 1$	0
$C = 2$	12 strings	$e^{-i\pi/4}$	-45
$C = 4$	$\|0101\rangle$, $\|1010\rangle$	$e^{-i\pi/2} = -i$	-90

All amplitudes remain 0.25 **after phase encoding.**

Case Study: After Mixing - Final Probabilities

State after Step 3 - $U(H_M, \beta = \pi/8)$:
$R_X(2\beta) = R_X(\pi/4)$ applied to each qubit mixes amplitudes across all bit strings. The differently-phased amplitudes from Step 2 now interfere:

$\|z\rangle$	$C(z)$	P	$\|z\rangle$	$C(z)$	P
$\|0000\rangle$	0	0.0067	$\|1000\rangle$	2	0.0455
$\|0001\rangle$	2	0.0455	$\|1001\rangle$	2	0.0638
$\|0010\rangle$	2	0.0455	$\|1010\rangle$	4	**0.1835**
$\|0011\rangle$	2	0.0638	$\|1011\rangle$	2	0.0455
$\|0100\rangle$	2	0.0455	$\|1100\rangle$	2	0.0638
$\|0101\rangle$	4	**0.1835**	$\|1101\rangle$	2	0.0455
$\|0110\rangle$	2	0.0638	$\|1110\rangle$	2	0.0455
$\|0111\rangle$	2	0.0455	$\|1111\rangle$	0	0.0067

The Classical Outer Loop: Finding Optimal Angles

The QAOA outer loop in practice:

1. Initialise (γ, β) randomly or from known good values
2. Run QAOA circuit $\rightarrow$ estimate $F_1(\gamma, \beta)$ from shots
3. Update angles using COBYLA or SPSA (gradient-free: the landscape is smooth at small p)
4. Repeat until convergence
5. Sample many bit strings from the final circuit; take the best as the solution

Why not parameter-shift here? Parameter-shift gives exact gradients but costs $2 \times 2p$ extra circuit runs. For $p = 1, 2$ on small problems, COBYLA converges faster with fewer circuit evaluations. At large p, gradient-based methods become necessary.

Applied Example: Weighted MaxCut

Suppose a communication network contains four servers $\{A, B, C, D\}$.
Each edge represents the communication bandwidth (Gbps) between two servers.

Objective: Partition the servers into two groups so that the total bandwidth crossing the partition is maximised.

Weighted Problem Hamiltonian

$$H_P = \sum_{(j,k) \in E} w_{jk} \frac{I - Z_j Z_k}{2}.$$

Circuit implementation: Every weighted edge is implemented exactly as before, CNOT $\rightarrow R_Z(2\gamma w_{jk}) \rightarrow$ CNOT, so only the rotation angle changes.

Key idea: Once the QAOA circuit has been built for MaxCut, extending it to weighted optimization problems requires only changing the Hamiltonian coefficients.

QAOA as a Hybrid Quantum-Classical Model

QAOA fits naturally into the hybrid variational framework studied in Session 11.

Hybrid Framework	QAOA	Interpretation
Trainable parameters	$(\gamma_1, \ldots, \gamma_p, \beta_1, \ldots, \beta_p)$	Only $2p$ parameters
Objective function	$-\langle H_P \rangle$	Equivalent to maximizing the expected cut value
Classical optimizer	COBYLA, SPSA, etc.	Updates circuit parameters
Quantum circuit	Evaluates objective	Returns expectation value
Final output	Sampled bit string	Best solution after many shots

How QAOA differs from VQCs

- The circuit structure is determined by the optimization problem.
- There is no learned feature encoder. The Hamiltonian defines the problem.
- Parameters are shared globally across each layer, not assigned to every qubit.
- The output is a sampled classical solution, rather than a prediction.

Table of Contents

Beyond MaxCut: Can QAOA Solve Other Problems?

MaxCut has given us one type of Problem Hamiltonian. Real applications need many others.

Problem	What the binary variable means
Job Scheduling	$x_{ij} = 1$ if job i is assigned to machine j
Portfolio Optimization	$x_i = 1$ if asset i is included in the portfolio
Knapsack	$x_i = 1$ if item i is packed
Vehicle Routing	$x_{ij} = 1$ if the route travels directly from stop i to stop j
Graph Coloring	$x_{ic} = 1$ if vertex i is assigned color c

Different meanings, different objectives, different constraints - but all of them decided by binary choices.

QUBO: A Common Language for Optimization

Every one of those problems reduces to the same mathematical object:

$$Q(x) = \sum_i c_i x_i + \sum_{i<j} c_{ij} x_i x_j + c_0, \qquad x_i \in \{0, 1\}$$

One linear coefficient per variable, one quadratic coefficient per pair - regardless of whether x_i meant "pack this item" or "assign this job to this machine."

QUBO = Quadratic Unconstrained Binary Optimization

- Binary decision variables (0/1)
- Quadratic objective function
- Constraints incorporated into the objective using penalty terms

Next: we build one of these, coefficient by coefficient, for a real problem.

Worked Example: A Small Project Selection Problem

A team must choose which projects to fund, on a limited budget.

Project	Score	Cost
Project 1	3	1
Project 2	5	2
Project 3	4	2

Budget = 3

Classical formulation:

$$\max\ 3x_1 + 5x_2 + 4x_3 \qquad \text{s.t.} \qquad x_1 + 2x_2 + 2x_3 \leq 3, \qquad x_i \in \{0, 1\}$$

This is small enough to check by hand - which is exactly why it is useful for learning the QUBO recipe before trusting software to do it.

Step 1: Remove the Inequality with a Slack Variable

QUBO has no notion of $\leq$. It can only minimize an unconstrained quadratic expression.

Fix: introduce a slack variable s that absorbs whatever budget is left unused, turning the inequality into an equality:

$$x_1 + 2x_2 + 2x_3 + s = 3, \qquad s \in \{0, 1, 2, 3\}$$

s itself is not binary - it has 4 possible values, so it must be binary-encoded:

$$s = s_1 + 2s_2, \qquad s_1, s_2 \in \{0, 1\}$$

Every inequality constraint silently adds extra qubits - one binary variable per slack bit. A 3-item problem with one budget constraint here needs $3 + 2 = 5$ binary variables, not 3.

Step 2: Build the Penalized QUBO

Flip the sign (QUBO minimizes) and **add a squared penalty** for any deviation from the equality constraint:

$$Q(x_1, x_2, x_3, s_1, s_2) = -(3x_1 + 5x_2 + 4x_3) + P(x_1 + 2x_2 + 2x_3 + s_1 + 2s_2 - 3)^2$$

Choosing P: it must make any constraint violation cost more than any possible gain in the objective.

- Largest possible score (funding everything) $= 3 + 5 + 4 = 12$
- Choose $P = 10$ - comfortably larger than any single-unit violation can be worth

Step 3: Expand into QUBO Coefficients

Expand the square. Since $x_i^2 = x_i$ for binary variables, every squared term collapses to linear, and cross-terms become the pairwise QUBO coefficients:

Linear terms

Term	Coeff.
x_1	-53
x_2	-85
x_3	-84
s_1	-50
s_2	-80

Quadratic terms

Term	Coeff.
x_1x_2, x_1x_3	40
x_2x_3	80
x_1s_1	20
x_2s_1, x_3s_1	40
x_1s_2	40
x_2s_2, x_3s_2	80
s_1s_2	40

Constant offset: 90

Nothing here is quantum yet - this is pure algebra.

Building H_P: Single-Qubit Terms

Every quadratic term touching qubit i feeds back into Z_i's coefficient, not just i's own linear term:

$$\text{coeff}(Z_i) = -\frac{c_i}{2} - \sum_j \frac{c_{ij}}{4}$$

Var.	$-c_i/2$	Quadratic terms touching i	$-\sum c_{ij}/4$	Total
x_1	26.5	$40+40+20+40 = 140$	-35	**-8.5**
x_2	42.5	$40+80+40+80 = 240$	-60	**-17.5**
x_3	42	$40+80+40+80 = 240$	-60	**-18**
s_1	25	$20+40+40+40 = 140$	-35	**-10**
s_2	40	$40+80+80+40 = 240$	-60	**-20**

A qubit coupled to more terms (like s_1, s_2) picks up a bigger correction - never read Z_i's coefficient off the QUBO row alone.

Building H_P: Two-Qubit Terms, and H_M

Each quadratic term also contributes one Z_iZ_j term, with coefficient $c_{ij}/4$:

$$c_{ij}\, x_i x_j \to \frac{c_{ij}}{4}\left(I - Z_i - Z_j + Z_iZ_j\right) \implies \text{coeff}(Z_iZ_j) = \frac{c_{ij}}{4}$$

Pair	c_{ij}	$c_{ij}/4$
x_1x_2	40	**10**
x_1x_3	40	**10**
x_1s_1	20	**5**
x_1s_2	40	**10**
x_2x_3	80	**20**

Pair	c_{ij}	$c_{ij}/4$
x_2s_1	40	**10**
x_2s_2	80	**20**
x_3s_1	40	**10**
x_3s_2	80	**20**
s_1s_2	40	**10**

Mixing Hamiltonian H_M - identical form for every QAOA problem:

$$H_M = \sum_{j=1}^{5} X_j = X_{x_1} + X_{x_2} + X_{x_3} + X_{s_1} + X_{s_2}$$

H_P and H_M for This Problem

Problem Hamiltonian H_P - 16 Pauli terms on 5 qubits:

Single-qubit terms

$$39\, I - 8.5\, Z_{x_1} - 17.5\, Z_{x_2} - 18\, Z_{x_3} - 10\, Z_{s_1} - 20\, Z_{s_2}$$

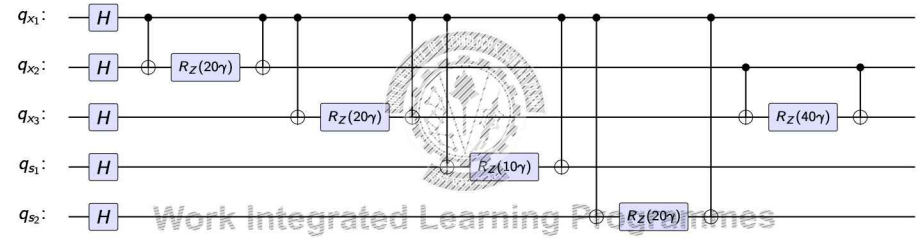
Two-qubit terms

$$10\, Z_{x_1}Z_{x_2} + 10\, Z_{x_1}Z_{x_3} + 5\, Z_{x_1}Z_{s_1} + 10\, Z_{x_1}Z_{s_2} + 20\, Z_{x_2}Z_{x_3} + 10\, Z_{x_2}Z_{s_1} + 20\, Z_{x_2}Z_{s_2} + 10\, Z_{x_3}Z_{s_1} + 20\, Z_{x_3}Z_{s_2} + 10\, Z_{s_1}Z_{s_2}$$

Circuit cost

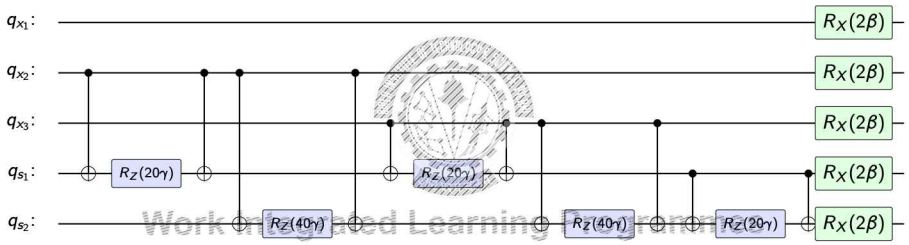
H_P needs 10 CNOT–R_Z–CNOT blocks per layer (one per ZZ term) - more than MaxCut's 4-edge cycle, since s_1, s_2 couple to every variable. H_M is always just n single-qubit R_X gates.

QAOA Circuit for H_P: Terms 1-5 of 10



Term i-j: CNOT(i,j) - $R_Z(2\gamma c_{ij})$ on qubit j - CNOT(i,j), using the H_P coefficients c_{ij} from two slides ago. 5 of 10 terms shown; continued next slide.

QAOA Circuit for H_P: Terms 6-10 of 10, and H_M



Terms 6-10 complete $U(H_P, \gamma)$; the final $R_X(2\beta)$ on every qubit is $U(H_M, \beta)$ - the mixer, unchanged from MaxCut. This whole two-slide circuit is *one* QAOA layer; it repeats p times.

The General Rule

1. Define one binary variable per decision
2. Write the objective as a linear function of those variables
3. Write each constraint as a linear inequality or equality
4. For every inequality, add a slack variable and binary-encode it
5. Combine: flip the objective's sign and add $P \times (\text{constraint})^2$, with P larger than the largest possible objective value
6. Expand using $x_i^2 = x_i$ to get the QUBO's coefficients
7. Convert to Ising ($x_i \to \frac{I - Z_i}{2}$), build the QAOA circuit, and check the result against a classical solve

This is the exact same recipe behind Knapsack, Scheduling, Portfolio Optimization, and Routing.

Building the QUBO: Hardcoded

Define the task as a QuadraticProgram:

```
qp = QuadraticProgram(name="ProjectSelection")
qp.binary_var(name="x1")
qp.binary_var(name="x2")
qp.binary_var(name="x3")

qp.maximize(linear={"x1": 3, "x2": 5, "x3": 4})
qp.linear_constraint(
    linear={"x1": 1, "x2": 2, "x3": 2},
    sense="<=", rhs=3, name="budget",
)

qubo = QuadraticProgramToQubo().convert(qp)
```

`QuadraticProgramToQubo().convert(qp)` performs *all* of Steps 1-3 automatically: adds the slack variable, binary-encodes it, builds the penalty term, and expands it into the 15 coefficients you derived by hand.

Building the QUBO: From a Dataset

For a real dataset, build the same three pieces from a DataFrame:

```
for item in df["item_id"]:
    qp.binary_var(name=item)

qp.maximize(linear=dict(zip(df["item_id"], df["value"])))
qp.linear_constraint(
    linear=dict(zip(df["item_id"], df["weight"])),
    sense="<=", rhs=capacity, name="capacity",
)

qubo = QuadraticProgramToQubo().convert(qp)
```

Same code, any size

Three projects or 200 warehouse items - the code is identical, only `df` changes.

Building H_P in Qiskit

One line replaces the entire hand-derivation from the previous slide:

```
operator, ising_offset = qubo.to_ising()

print(f"Number of qubits: {operator.num_qubits}")
print(f"Ising offset: {ising_offset:.4f}")
print(f"H_P has {len(operator)} Pauli terms")
```

Seeing the actual Pauli terms - your H_P table, printed:

```
for pauli, coeff in zip(operator.paulis, operator.coeffs):
    print(f"{coeff.real:8.4f} * {pauli}")
```

Where is H_M?

It never appears in this code. Qiskit's `QAOA` class hard-codes the standard X-mixer internally - you never build it, because it is always $\sum_j X_j$, regardless of the problem.

Building and Running QAOA in Qiskit

Configure the circuit - $p = 2$ from the slides is `reps=2`:

```
sampler = Sampler()
optimizer = COBYLA(maxiter=250)

qaoa = QAOA(
    sampler=sampler,
    optimizer=optimizer,
    reps=2,               # QAOA depth p
    initial_point=None,
)

qaoa_solver = MinimumEigenOptimizer(qaoa)
```

Solve - internally calls `to_ising()` again, so H_P is usually invisible in practice:

```
qaoa_result = qaoa_solver.solve(qubo)
print(qaoa_result.prettyprint())
```


How Is QAOA Applied in Practice?

The six-step recipe is rarely done by hand at full scale - software libraries automate it:

Library	What it actually gives you
Qiskit Optimization	`QuadraticProgram` → `QuadraticProgramToQubo` → `MinimumEigenOptimizer` - exactly our lab notebook
PennyLane	`qml.qaoa` - differentiable QAOA circuits, gradient-based angle optimization
D-Wave Ocean SDK	Binary Quadratic Model (BQM) format; quantum annealing hardware, not gate-based QAOA

Table of Contents

Which Architecture Should I Choose?

Every architecture in Module 7 answers a different problem. The choice follows from the problem, not from preference.

Problem type	Architecture	Why
Classical images or signals	CNN encoder + HEA	CNN compresses $D \to n$; HEA classifies in Hilbert space; standard hybrid pipeline (Session 11)
Classical tabular data, small N	MLP encoder + HEA	No spatial structure, so no CNN needed; same series hybrid, different encoder
Quantum state input	QCNN	Input already quantum; hierarchical structure; $O(\log N)$ params; provably no barren plateau
Combinatorial optimi…	QAOA	Problem encodes directly as…

Practical Guidelines for Designing Hybrid QNNs

In order of application - follow this sequence:

1	**Compress features first**	Reduce classical input to qubit count n *before* thinking about the quantum circuit. If no good compression exists, the pipeline will not work.
2	**Start shallow**	Begin with 2–3 variational layers. Check gradient norms before adding depth. Depth is the fastest route to barren plateaus on NISQ.
3	**Use local cost functions**	Measure 1–2 output qubits, not all n. Cerezo et al. (2021): local cost changes gradient decay from exponential to polynomial.
4	**Prefer problem-structured circuits**	HEA is a last resort, not a default. Use QCNN for hierarchical data, QAOA for combinatorics, re-uploading for ex…

Table of Contents

Practice Problem 1 - From Scheduling to QAOA

Scenario

A company has four jobs $\{J_1, J_2, J_3, J_4\}$ that must be assigned to two identical servers. The objective is to balance the workload between the servers.

Design a QAOA solution by answering the following:

1. Define suitable binary decision variables.
2. Explain how the scheduling objective can be written as an Ising Hamiltonian. What do the coefficients J_{ij} and h_i represent?
3. Sketch the QAOA pipeline:

$$\boxed{\text{Problem}} \to \boxed{H_P} \to \boxed{\text{QAOA Circuit}} \to \boxed{\gamma, \beta \text{ optimization}} \to \boxed{\text{Best schedule}}$$

When would QAOA become preferable?

Practice Problem 2 - Choosing the Right Architecture

For each application, choose the most suitable architecture.

Application	Architecture	Reason
MRI image classification		
Quantum state classification		
Vehicle routing		
Customer churn prediction		
Portfolio optimisation		

Possible architectures CNN/MLP + HEA; QCNN; QAOA

Practice Problem 3

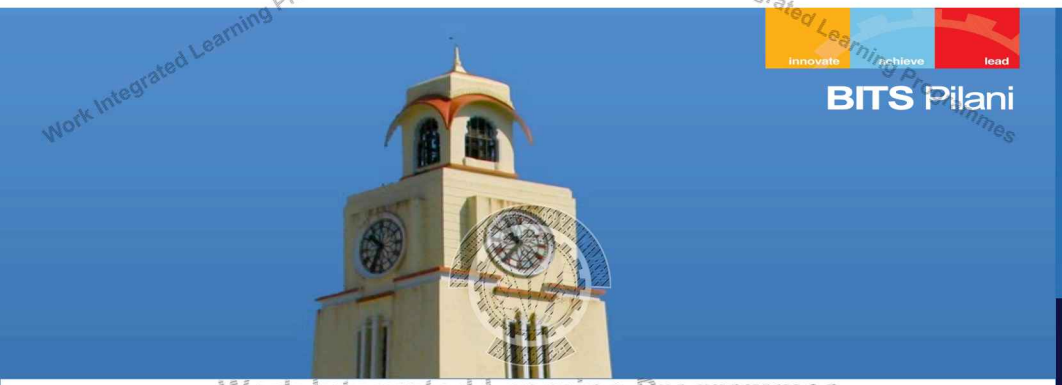
A team must choose which projects to fund from a shortlist, given a fixed budget.

Project	Score	Cost
Project A	3	1
Project B	5	2
Project C	4	2

Budget = 3

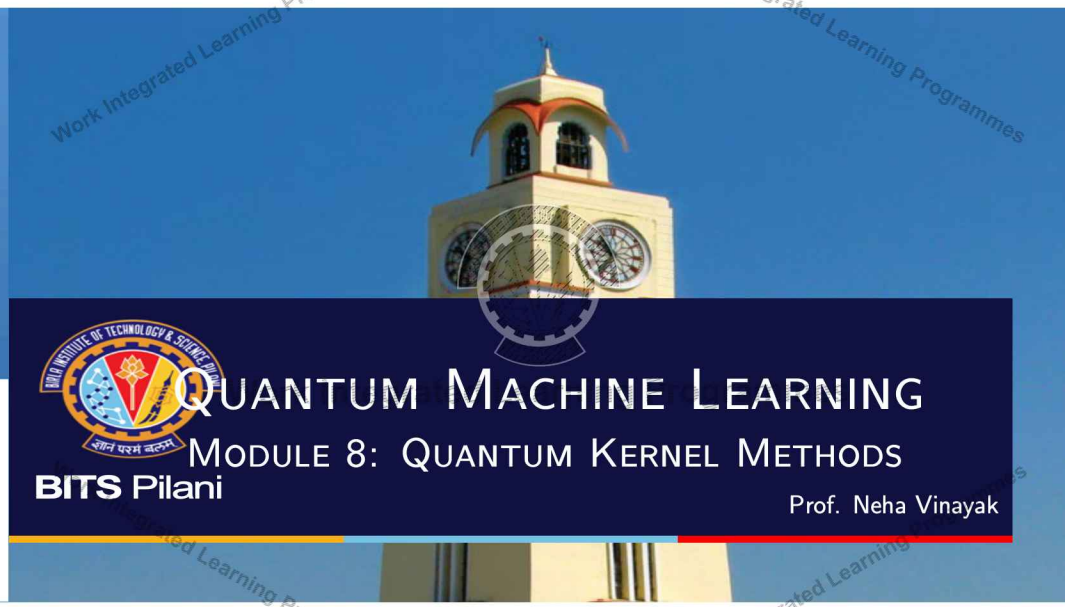
Answer the following:

1. Define binary decision variables and write the classical objective and constraint. Explain why a slack variable is needed here, and binary-encode it.
2. Write the fully expanded QUBO (justify your choice of penalty weight P).
3. State which projects should be funded, and verify your answer without using the QUBO.
4. Convert the QUBO into the Ising Problem Hamiltonian H_P. Give every single-qubit and two-qubit Pauli coefficient.
5. State the Mixing Hamiltonian H_M. How many `CNOT`-R_Z-`CNOT` blocks does one QAOA layer need to implement $U(H_P, \gamma)$ for this problem, and why that many?



BITS Pilani

Thank you :)



Quantum Machine Learning

Module 8: Quantum Kernel Methods

BITS Pilani

Prof. Neha Vinayak

Where We Are

- Module 6 - Variational Quantum Circuits: trainable ansatz $\mathcal{G}_\theta$, parameter-shift gradients, barren plateaus
- Module 7 - Hybrid Quantum-Classical Models: QNN pipelines, NISQ constraints, QAOA
- **Module 8 - Quantum Kernel Methods**: the *same* encoded states $\rho(x)$, viewed not as inputs to a trainable circuit, but as feature vectors in a similarity-based classical learner

Today's central claim (Schuld & Petruccione, Ch. 6)

Most supervised, deterministic quantum models can be reformulated as a *classical kernel method* whose kernel is computed by a quantum computer.

Recap: Classical Kernel Methods

Definition 2.7 (Kernel)

A kernel is a positive semi-definite bivariate function $\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$. For any $D = \{x_1, \ldots, x_M\} \subset \mathcal{X}$, the Gram matrix $K_{m,m'} = \kappa(x_m, x_{m'})$ is positive semi-definite.

- Every kernel can be written as an inner product in some feature space: $\kappa(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{F}}$
- **Kernel trick:** we never need to compute $\phi(x)$ explicitly - only $\kappa(x, x')$
- Classical examples: linear $x^T x'$, polynomial $(x^T x' + c)^p$, Gaussian $e^{-\gamma \|x - x'\|^2}$

This is precisely the structure we will exploit: a quantum computer can evaluate a kernel that may be classically hard to compute, even without ever writing down $\phi(x)$.

Why Feature Maps Matter: The XOR Problem

Data: $D = \{(\pm 1, \pm 1)\}$, label = product of the two coordinates

- Not linearly separable in $\mathbb{R}^2$
- Feature map $\phi(x_1, x_2) = (x_1, x_2, x_1 x_2)$ adds the product term
- In the new space, a hyperplane separates the classes

class $+1$ (circles) $\Rightarrow x_1 x_2 = +1$ | class -1 (squares) $\Rightarrow x_1 x_2 = -1$

Same logic applies to quantum embeddings.

The Quantum-Classical Bridge

- Quantum computing and kernel methods share a structural similarity: both map information into *inaccessibly large spaces* and process it only through inner products
- Kernel methods: access the feature space $\mathcal{F}$ via $\kappa(x, x') = \langle \phi(x), \phi(x') \rangle$
- Quantum computing: access the Hilbert space $\mathcal{H}$ via measurement, expressible as inner products of quantum states $\langle \psi | \phi \rangle$

Key consequence

Training a quantum model is equivalent to training a kernel method with the *quantum kernel*. The expressivity, optimisation, and generalisation of a quantum model are largely fixed by its data-encoding strategy.

Table of Contents

The Data-Encoding Feature Map

A data-encoding circuit $S(x)$ prepares $S(x)|0\rangle = |\phi(x)\rangle$.
$x \mapsto |\phi(x)\rangle$ can't be used directly as the feature map -quantum models are not linear in the state vector.
Measurement outcomes are linear in the *density matrix*.

Data-encoding feature map

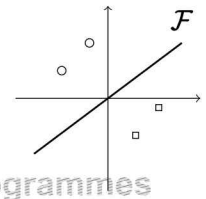
Let $\mathcal{F}$ be the space of $2^n \times 2^n$ complex matrices with inner product $\langle \rho, \sigma \rangle_{\mathcal{F}} = \mathrm{tr}\{\rho^\dagger \sigma\}$. The feature map is

$$\phi : \mathcal{X} \to \mathcal{F}, \qquad \phi(x) = |\phi(x)\rangle\langle\phi(x)| = \rho(x)$$

Quantum Models as Linear Classifiers

Linear model in feature space

For a feature map $\phi : \mathcal{X} \to \mathcal{F}$,
any $f(x) = \langle \phi(x), w \rangle_{\mathcal{F}}$
with $w \in \mathcal{F}$ is a **linear model** in $\mathcal{F}$.



In a deterministic quantum model
$f(x) = \mathrm{tr}\{M\rho(x)\}$
the measurement operator M is the weight w.
So M defines a linear decision boundary in $\mathcal{F}$.

decision boundary $= \mathrm{tr}\{M\rho(x)\} = 0$

Table of Contents

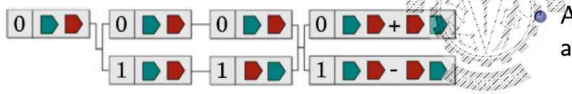
The Quantum Kernel

Let ϕ be a data-encoding feature map over $\mathcal{X}$. The quantum kernel is

$$\kappa(x, x') = \mathrm{tr}\{\rho(x)\rho(x')\} = |\langle\phi(x)|\phi(x')\rangle|^2$$

- This is the inner product of two data-encoding feature vectors - exactly the quantity a quantum computer can estimate by *measurement*
- **Why it is a valid kernel:** κ is the product of a complex kernel $\kappa_c(x, x') = \langle\phi(x)|\phi(x')\rangle$ and its conjugate; products of kernels are kernels, and the conjugate of a kernel is positive semi-definite.
- So $\kappa(x, x') = |\langle\phi(x)|\phi(x')\rangle|^2$ is guaranteed positive semi-definite - it is a legitimate kernel for classical kernel theory

Estimating the Kernel: The Swap Test



- Ancilla in superposition controls a SWAP between the two registers
- Acceptance probability of measuring ancilla in $|0\rangle$:

$$p_0 = \tfrac{1}{2} + \tfrac{1}{2}\,\mathrm{tr}\{\rho(x)\rho(x')\}$$

- Hence: $\kappa(x, x') = 2p_0 - 1$

Cost: 1 ancilla + two full registers ($2n + 1$ qubits), $O(n)$ gates.

Swap Test: Where the Formula Comes From

Ancilla $|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, then controlled-SWAP on the two registers:

$$\tfrac{1}{\sqrt{2}}\big(\,|0\rangle\,|\phi(x)\rangle\,|\phi(x')\rangle + |1\rangle\,|\phi(x')\rangle\,|\phi(x)\rangle\,\big)$$

A second Hadamard on the ancilla interferes the two branches:

$$|\psi\rangle = \tfrac{1}{2}\,|0\rangle \otimes \big(\,|\phi(x)\phi(x')\rangle + |\phi(x')\phi(x)\rangle\,\big) + \tfrac{1}{2}\,|1\rangle \otimes \big(\,|\phi(x)\phi(x')\rangle - |\phi(x')\phi(x)\rangle\,\big)$$

The probability of measuring the ancilla in $|0\rangle$ follows from expanding the norm of the symmetric branch:

$$p_0 = \big\|\,\tfrac{1}{2}\big(\,|\phi(x)\phi(x')\rangle + |\phi(x')\phi(x)\rangle\,\big)\big\|^2 = \tfrac{1}{2} + \tfrac{1}{2}|\langle\phi(x)|\phi(x')\rangle|^2$$

Sanity check: if $x = x'$, the swap does nothing and the circuit reduces to H then H on the ancilla - it must return to $|0\rangle$ with certainty. Indeed $p_0 = \frac{1}{2} + \frac{1}{2}(1) = 1$. ✓

Estimating the Kernel: The Inversion Test

Run $S^\dagger(x')S(x)\,|0\rangle$ and measure the probability of observing the all-zeros outcome:

$$\kappa(x, x') = \left|\langle 0|\, S^\dagger(x')S(x)\,|0\rangle\right|^2$$

- Needs only n qubits (vs. $2n + 1$ for the swap test) - but requires implementing $S^\dagger(x')$
- Many encoding circuits are self-inverting up to a sign (e.g. $R(\theta)^\dagger = R(-\theta)$), making this cheap in practice
- This is the routine used for angle-/Pauli-rotation feature maps, the encoding family we use throughout this course

Numerical — Toy Dataset and Encoding

Setup: 1-qubit angle encoding, $S(x) = R_y(x)$, three points

$$x_1 = 0, \qquad x_2 = \frac{\pi}{2}, \qquad x_3 = \pi$$

$$|\phi(x)\rangle = R_y(x)|0\rangle = \cos\tfrac{x}{2}|0\rangle + \sin\tfrac{x}{2}|1\rangle$$

- $|\phi(x_1)\rangle = |0\rangle$
- $|\phi(x_2)\rangle = \frac{1}{\sqrt{2}}\big(|0\rangle + |1\rangle\big)$
- $|\phi(x_3)\rangle = |1\rangle$

Goal: compute the 3×3 *kernel matrix* $K_{ij} = \kappa(x_i, x_j) = |\langle\phi(x_i)|\phi(x_j)\rangle|^2$ *by hand.*

Numerical — Computing $\kappa(x, x')$ by Hand

$$\langle\phi(x_1)|\phi(x_2)\rangle = \langle 0|\Big(\tfrac{1}{\sqrt{2}}|0\rangle + \tfrac{1}{\sqrt{2}}|1\rangle\Big) = \tfrac{1}{\sqrt{2}} \Rightarrow \kappa(x_1, x_2) = \tfrac{1}{2}$$

$$\langle\phi(x_1)|\phi(x_3)\rangle = \langle 0|1\rangle = 0 \Rightarrow \kappa(x_1, x_3) = 0$$

$$\langle\phi(x_2)|\phi(x_3)\rangle = \tfrac{1}{\sqrt{2}}\langle 0|1\rangle + \tfrac{1}{\sqrt{2}}\langle 1|1\rangle = \tfrac{1}{\sqrt{2}} \Rightarrow \kappa(x_2, x_3) = \tfrac{1}{2}$$

$$\kappa(x_i, x_i) = 1 \quad \text{for all } i$$

Check against the closed form for angle encoding (Table 6.1): $\kappa(x, x') = |\cos\big(\frac{x - x'}{2}\big)|^2$.

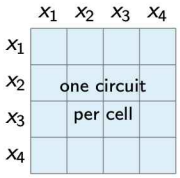
$$\kappa(0, \pi/2) = \cos^2(\pi/4) = \tfrac{1}{2}\ \checkmark \qquad \kappa(0, \pi) = \cos^2(\pi/2) = 0\ \checkmark$$

Numerical — The Kernel (Gram) Matrix

$$K = \begin{pmatrix} \kappa(x_1, x_1) & \kappa(x_1, x_2) & \kappa(x_1, x_3) \\ \kappa(x_2, x_1) & \kappa(x_2, x_2) & \kappa(x_2, x_3) \\ \kappa(x_3, x_1) & \kappa(x_3, x_2) & \kappa(x_3, x_3) \end{pmatrix} = \begin{pmatrix} 1 & 0.5 & 0 \\ 0.5 & 1 & 0.5 \\ 0 & 0.5 & 1 \end{pmatrix}$$

- Symmetric, positive semi-definite (as guaranteed by Definition 6.2) - check: eigenvalues are $\{2, 1, 0\}$, all ≥ 0
- x_1 and x_3 have zero kernel value - they are mapped to *orthogonal* states ($|0\rangle$ vs. $|1\rangle$), maximally dissimilar
- This 3×3 matrix is exactly what a classical SVM solver needs as input - it never sees x or $\phi(x)$, only K

How the Gram Matrix Is Actually Built



- Every entry $K_{m,m'} = \kappa(x_m, x_{m'})$ is **not** a formula lookup - it is a separate swap-test or inversion-test circuit execution
- For M training samples, the Gram matrix has M^2 entries (roughly $M^2/2$ after using symmetry)

Consequence

Building the full kernel matrix requires $O(M^2)$ quantum circuit executions - this is exactly what makes the scaling comparison in §8.5 meaningful.

Examples of Quantum Kernels

Encoding	Kernel $\kappa(x, x')$
Basis encoding	$\delta_{x,x'}$
Amplitude encoding	$\lvert x^\dagger x' \rvert^2$
Repeated amplitude encoding ($r\times$)	$(\lvert x^\dagger x' \rvert^2)^r$
Angle encoding	$\lvert \prod_{k=1}^{N} \lvert \cos(x_k - x'_k) \rvert^2$
Coherent state encoding	$e^{-\lvert x-x' \rvert^2}$
General time-evolution encoding	$\sum_{s,t\in\Omega} e^{isx} e^{-itx'} c_{s,t}$

- Notice: amplitude encoding gives the (squared) **polynomial kernel**; coherent-state encoding gives the **Gaussian kernel** - both classically familiar
- **Basis encoding gives a Kronecker delta** - every distinct input is orthogonal to every other.

Fourier Structure of Quantum Kernels

Fourier representation

For data encoded via e^{-ix_iG} gates with generator G (eigenvalues $\lambda_1, \ldots, \lambda_d$), interleaved with arbitrary unitaries, the quantum kernel takes the form

$$\kappa(x, x') = \sum_{s,t\in\Omega} e^{isx} e^{-itx'} c_{s,t}, \qquad \Omega \subseteq \mathbb{R}^N$$

- For Pauli generators with integer eigenvalue gaps, this collapses to a genuine multi-dimensional **Fourier series**
- **Repeating** the encoding (data re-uploading, Module 7) increases the number of available frequencies ⇒ a less smooth, more expressive kernel

The data-encoding strategy doesn't just affect circuit depth - it literally chooses the Fourier basis your model can express.

Why Bother? Classically Hard Kernels

- If every quantum kernel could be evaluated efficiently on a classical computer, the kernel perspective would be mathematically elegant but practically pointless
- It has been shown that there exist data-encoding feature maps whose kernel *cannot* be efficiently estimated classically - the swap/inversion test gives a quantum computer access to inner products in a space too large to simulate
- This is the precise sense in which a quantum kernel can offer something beyond the classical kernels (linear, polynomial, Gaussian, ...).

Classical vs Quantum Kernels

Step	Classical Kernel	Quantum Kernel
Input	Dataset	Dataset
Feature Map	Classical feature map $\phi(x)$	Quantum feature map $\lvert\phi(x)\rangle$
Similarity Computation	Classical kernel formula $k(x, x')$	Quantum circuit Swap Test / Inversion Test
Output	Gram Matrix K	Gram Matrix K
Learning Algorithm	Classical SVM	Classical SVM

Only one step changes: The learning algorithm remains identical in both cases. The quantum computer is used only to estimate the entries of the Gram matrix.

Table of Contents

Why Study the RKHS?

- We now have two feature spaces with the *same* kernel: $\mathcal{F}$ (density matrices) and a new one built purely from κ itself
- This second space - the **Reproducing Kernel Hilbert Space (RKHS)** - turns out to contain exactly the quantum model functions f
- Why bother? Because RKHS theory lets us answer questions about *expressivity*, *optimal training*, and *regularisation* purely from the kernel

Roadmap for this subsection

$\kappa \longrightarrow$ RKHS $\mathcal{F}_\kappa \longrightarrow$ quantum models = functions in $\mathcal{F}_\kappa \longrightarrow$ training = optimising over $\mathcal{F}_\kappa$ (§8.4)

Constructing the RKHS

Step 1: Build a similarity function from one training point $\kappa(x_i, x)$

- Fix one training point x_i
- For every new input x, the function $\kappa(x_i, x)$ returns its similarity to x_i

Step 2: Combine many similarity functions

$$\boxed{f(x) = \sum_i \alpha_i \kappa(x_i, x)}$$

- Each training point contributes one similarity function
- α_i decides how much importance (weight) to give that function
- The RKHS consists of **all such weighted combinations**

Comparing Functions in the RKHS

Suppose we build two functions:

$$f(x) = \sum_i \alpha_i \kappa(x_i, x), \qquad g(x) = \sum_j \beta_j \kappa(x_j, x).$$

To measure how similar these two functions are, we define the inner product

$$\boxed{\langle f, g \rangle_{\mathcal{F}_\kappa} = \sum_{i,j} \alpha_i \beta_j \kappa(x_i, x_j)}$$

- Just like vectors, functions also need an inner product
- The remarkable part is that this depends only on the kernel values
- No quantum circuits, density matrices, or feature maps appear anymore

Connection Between RKHS & Quantum Models

Theorem

The functions represented by the RKHS are exactly the same functions represented by the quantum model

$$f(x) = \mathrm{tr}\{M\rho(x)\}.$$

Every function in the RKHS corresponds to a quantum model, and every quantum model corresponds to a function in the RKHS.

- The RKHS does **not** define a new learning algorithm
- It is simply another mathematical description of the same quantum model
- This lets us analyse the model using only the kernel

How Expressive Is the Kernel?

Universal Kernel

A kernel is called **universal** if the functions in its RKHS can approximate any continuous target function as accurately as desired.

Universal Kernel $\Longrightarrow$ Highly Expressive Function Space

- Different inputs should map to different quantum states (injective feature map)
- If two different inputs produce the same quantum state, the model cannot distinguish them
- Therefore, universal approximation becomes impossible

When Universality Breaks

Example: Single-qubit Pauli rotation encoding

$$S(x) = e^{ix\sigma}$$

Since the encoding is 2π-periodic, $\rho(x) = \rho(x + 2\pi)$

- Different inputs are mapped to the **same quantum state**
- The model cannot distinguish between them
- Therefore, some target functions can never be represented
- Hence, the kernel is **not universal** over domains wider than 2π

Connection to the Fourier viewpoint

The encoding determines the accessible frequencies and determines the family of functions the model can represent.

Table of Contents

How Does a Quantum Kernel Learn?

Question: How do we actually train the model?

Unlike a Variational Quantum Circuit (VQC), there are **no trainable quantum gates** The quantum encoding circuit is fixed in both.

The learning objective is

$$\min_{f \in \mathcal{F}_\kappa} \underbrace{\hat{R}_L(f)}_{\text{Training Error}} + \lambda \underbrace{\|f\|^2_{\mathcal{F}_\kappa}}_{\text{Regularization}}$$

where

- $\hat{R}_L(f)$: empirical training error (loss on the training set)
- $\|f\|^2_{\mathcal{F}_\kappa}$: model complexity
- λ : regularization parameter controlling the trade-off

The Representer Theorem

Representer Theorem

The optimal prediction function always has the form

$$f_{opt}(x) = \sum_{m=1}^{M} \alpha_m \kappa(x_m, x)$$

where $\alpha_1, \alpha_2, \ldots, \alpha_M$ are real numbers learned during training.

- We only combine similarity functions centred at the training samples
- Infinite-dimensional optimisation becomes optimisation over only M coefficients

What Does the SVM Actually Learn?

$$f(x) = \sum_{m=1}^{M} \alpha_m \kappa(x_m, x)$$

Prediction for a new sample:

1. Compute similarity with every training point

$$\kappa(x_1, x),\ \kappa(x_2, x), \ldots, \kappa(x_M, x)$$

2. Multiply each similarity by its learned weight α_m
3. Add them together to obtain $f(x)$

Training Becomes a Convex Optimization

Using the Representer Theorem,
the optimisation reduces to

$$\min_{\alpha \in \mathbb{R}^M} \frac{1}{M} \sum_m L + \lambda \alpha^T K \alpha$$

where

- K is the Gram matrix
- α contains the coefficients to be learned

Key Result

If the loss function is convex,
the optimisation has

- one unique global minimum
- no local minima
- no barren plateaus

The Quantum Support Vector Machine

Choosing the hinge loss gives the familiar SVM optimisation

$$\max_{\alpha} \sum_{m} \alpha_m - \frac{1}{2} \sum_{m,m'} \alpha_m \alpha_{m'} y_m y_{m'} \kappa(x_m, x_{m'})$$

Prediction for a new sample

$$\boxed{\hat{y}(x) = \text{sign}\left(\sum_{m} \alpha_m y_m \kappa(x_m, x) \right)}$$

Key Idea

The QSVM is simply a classical SVM that uses a quantum-computed kernel.

Setting up the QSVM

Training data $x_1 = 0,\ y_1 = -1 \qquad x_2 = \frac{\pi}{2},\ y_2 = +1 \qquad x_3 = \pi,\ y_3 = -1$

The kernel matrix (computed earlier) is

$$K = \begin{pmatrix} 1 & 0.5 & 0 \\ 0.5 & 1 & 0.5 \\ 0 & 0.5 & 1 \end{pmatrix}$$

Objective function:

$$\max_{\alpha} \sum_{m} \alpha_m - \frac{1}{2} \sum_{m,m'} \alpha_m \alpha_{m'} y_m y_{m'} K_{m,m'}$$

subject to $\sum_m \alpha_m y_m = 0, \qquad \alpha_m \geq 0.$

What Is the QSVM Actually Learning?

- Number of optimisation variables = number of training samples

$$M = 3 \implies \alpha_1, \alpha_2, \alpha_3$$

- The quantum kernel matrix K is already known after the quantum circuit finishes.
- The class labels y are also known from the training data.
- The only unknowns are the coefficients

$$\alpha_1, \alpha_2, \ldots, \alpha_M,$$

which are learnt by the classical SVM.

- After training, only the training samples with

$$\boxed{\alpha_m > 0}$$

become the **support vectors** and contribute to prediction.

Key Takeaway

The quantum computer computes the kernel matrix. The classical SVM learns the coefficients α. Together, they define the final classifier.

Making Predictions

The trained classifier is $f(x) = -2\kappa(x_1, x) + 4\kappa(x_2, x) - 2\kappa(x_3, x)$.
Evaluate on the training points:

$$f(0) = 0,$$

$$f\left(\frac{\pi}{2}\right) = -2(0.5) + 4(1) - 2(0.5) = 2 > 0,$$

$$f(\pi) = 0.$$

Observation

The second point is classified as the positive class, while the two boundary points become support vectors.

Practice Problem

Consider the following training set: $x_1,\ x_2,\ x_3, \qquad y = (-1, +1, -1)$ with quantum kernel matrix

$$K = \begin{pmatrix} 1 & 0.4 & 0 \\ 0.4 & 1 & 0.4 \\ 0 & 0.4 & 1 \end{pmatrix}.$$

Suppose the trained QSVM learns $\alpha = (1,\ 2,\ 1)$

Answer the following:

1. Write the matrix $Q_{ij} = y_i y_j K_{ij}$.
2. Write the QSVM decision function $f(x) = \sum_{m=1}^{3} \alpha_m y_m \kappa(x_m, x)$.
3. Which training samples are the support vectors?
4. Explain why the quantum computer is no longer required after the Gram matrix has been constructed.

Table of Contents

Two Ways to Learn from Quantum Data

Variational Learning	**Kernel Learning**
Encoding Circuit	Encoding Circuit
↓	↓
Trainable Quantum Circuit	Quantum Kernel Matrix
↓	↓
Gradient Optimisation	Classical Convex Optimisation
↓	↓
Prediction	Prediction

Observation

Both approaches start with the same quantum encoding. The difference lies entirely in *how learning is performed*.

Two Ways to Search the Same Space

Kernel-based training

- Optimises $\alpha \in \mathbb{R}^M$ - the subspace spanned by the M training points
- This subspace is *guaranteed* to contain M_{opt} (representer theorem)
- Convex ⇒ finds the true global optimum

Variational training (Module 6)

- Optimises $\theta \in \mathbb{R}^K$ - the subspace explored by a chosen ansatz $\mathcal{G}_\theta$
- This subspace may *not* contain M_{opt} at all
- Non-convex ⇒ no global-optimum guarantee, barren plateaus possible

Guarantee

Kernel-based training finds a measurement at least as good as the best variational result - at the cost of computing pairwise kernel values.

The Scaling Trade-off

	Kernel-based	Variational
Circuits to evaluate	$O(M^2)$ - full Gram matrix	$O(\|\theta\|M)$ - parameter-shift rule
Optimisation landscape	convex, unique minimum	non-convex, local minima
Ansatz design needed?	no	yes
Barren plateaus?	not applicable	a real risk (Module 6)

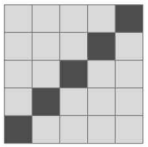
- If $|\theta|$ grows slowly with M, variational training approaches the favourable $O(M)$ scaling of classical neural networks - kernel-based training cannot
- If $|\theta|$ grows linearly with M (as in many practical ansätze), both methods are $O(M^2)$ - and in that regime, the kernel route wins on guarantees
- **Reported practical finding:** for ∼10–20 parameters and ∼100 samples, kernel-based training is often faster in wall-clock time once hardware gradient overhead is included

Table of Contents

When a Kernel Overfits

Basis encoding revisited: $\kappa(x, x') = \delta_{x,x'}$

- Gram matrix is the identity - every pair of distinct points has zero similarity
- An SVM can still drive *training* error to zero (each point gets its own α_m as a lookup weight)
- But it has learned nothing transferable: every new test point is equally (dis)similar to all training points



identity Gram matrix

In RKHS terms: the space of functions built from delta-bumps is maximally non-smooth - exactly the kind of function class that memorises rather than generalises.

A Good Encoding Clusters Classes

- A useful encoding maps same-class points close together and different-class points far apart in feature space
- Formally: draw $\rho(c)$ for class C, define the class "mixture" $\sigma_C = \sum_{c \in C} p(c)\rho(c)$
- Tight intra-class clustering $\Rightarrow$ σ_C has low rank; large inter-class distance $\Rightarrow$ $\mathrm{tr}\{\sigma_C \sigma_{C'}\}$ is small

Practical regularisation levers (kernel-based view)

- Tune the regularisation strength λ - it penalises $\|f\|_{\mathcal{F}_\kappa}$
- Control encoding repetitions / Fourier bandwidth - fewer repetitions $\Rightarrow$ smoother kernel $\Rightarrow$ less overfitting risk, at the cost of expressivity
- Avoid encodings (like raw basis encoding)

When Should I Use a Quantum Kernel?

Use Quantum Kernel	Use VQC
Small datasets	Large datasets
Need global optimum	Need trainable representations
Fixed feature map	Learnable feature map
Avoid barren plateaus	Can exploit optimisation
Classical SVM backend	End-to-end quantum model

One sentence to remember

Quantum kernels replace optimisation with similarity computation.

Table of Contents

Module 8 Summary

1 Data-encoding feature map $\phi(x) = \rho(x)$; quantum models are linear in this space
2 Quantum kernel $\kappa(x, x') = |\langle\phi(x)|\phi(x')\rangle|^2$; estimated via swap / inversion test
3 RKHS of $\kappa \equiv$ space of quantum models; encoding fixes expressivity (Fourier view)
4 Representer theorem $\Rightarrow$ training is a finite, convex program; QSVM as a special cas
5 Kernel-based: global optimum, $O(M^2)$; Variational: no guarantee, $O(|\theta|M)$
6 Encoding choice fixes regularisation; basis encoding overfits, smooth kernels general
7 Clustering and combinatorial ML reuse the same Gram-matrix machinery

Next: Post-2021 developments - exponential concentration of quantum kernels, projected kernels, and bandwidth/inductive-bias results

Why Are Quantum Kernels Still Challenging?

Quantum kernels have

- no barren plateaus,
- convex optimisation,
- guaranteed global optimum.

So why aren't they replacing classical kernels everywhere?

Answer

The challenge is no longer the optimisation.
The challenge is designing a **good quantum kernel** that continues to work as problems become larger.

Research Frontier — Exponential Concentration

- As the number of qubits or circuit depth increases, all kernel values become almost identical.
- The Gram matrix becomes nearly constant.
- The SVM now has very little information to learn from.

Key Idea

Earlier, gradients vanished in VQCs.
Here, **similarities vanish** in quantum kernels.
This phenomenon is called **Exponential Concentration**.

Reference: Thanasilp et al., Nature Communications (2024)

Research Frontier — How Do We Fix It?

1. **Projected Quantum Kernels**
   - Compare only small parts of the quantum state.
   - Ignore information that causes concentration.
2. **Better Bandwidth**
   - Too small ⇒ memorisation.
   - Too large ⇒ underfitting.
   - Choose the right balance.
3. **Symmetry-Aware Encodings**
   - Build known symmetries directly into the feature map.

Research Frontier — Running on Today's Hardware

Can quantum kernels run on today's quantum computers?

Yes, but with practical challenges.

- Finite-shot measurement noise.
- Gate errors and decoherence.
- Noisy Gram matrices.

Researchers improve performance by

- increasing measurement shots,
- regularising noisy kernel matrices,
- using projected quantum kernels.

Research Frontier — The Current Picture

- Quantum kernels are **not automatically better** than classical kernels.
- Choosing the right encoding is the key research problem.
- Advantages are expected mainly for structured problems with useful symmetries.

Take-home Message

The question today is no longer
"Can a quantum computer compute a kernel?"
It is
"When does that kernel outperform the best classical alternative?"

BITS Pilani

Thank you :)

Quantum Machine Learning

Module 9: Quantum Reinforcement Learning

BITS Pilani

Prof. Neha Vinayak

Table of Contents

Reinforcement Learning Recap

Before we begin Quantum Reinforcement Learning...

You have already completed a full course on Deep Reinforcement Learning. Today's goal is **not** to revisit RL, but to recall only the concepts needed to understand how quantum circuits fit into existing DRL algorithms.

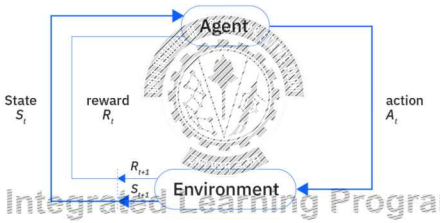
- Agent–environment interaction
- Why Deep RL was introduced
- Value-based vs. policy-based learning
- Where the neural network fits into the pipeline

Today's question:

What changes if we replace the neural network with a Parameterized Quantum Circuit (PQC)?

The Agent–Environment Interaction

Every reinforcement learning algorithm follows the same interaction loop.



Observe State s_t ⟶ **Choose Action** a_t ⟶ **Receive Reward & Next State** (r_{t+1}, s_{t+1})

Why Deep Reinforcement Learning?

Small Problems

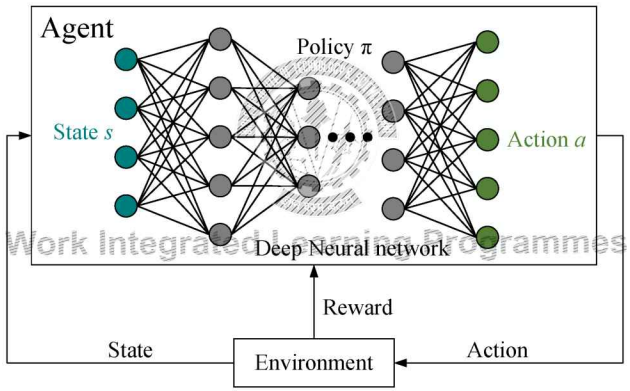
Grid World
⇓
Tabular Q-Learning
⇓
Store every $Q(s, a)$

Real Applications

Autonomous Driving
Robotics
Game Playing
Millions of possible states
⇓
Q-table becomes impossible

Deep RL = Classical RL + Neural Network Function Approximation

Deep RL



Deep Q-Networks: Replacing the Q-Table

Current State s_t ⟹ Neural Network Q_θ ⟹ Q-values for all Actions $\{Q(s, a)\}$

- The neural network approximates the Q-function instead of storing a large Q-table.
- Replay buffer and target network are additional components that improve training stability.

This neural network is the component we will replace with a PQC.

Two Families of Deep Reinforcement Learning

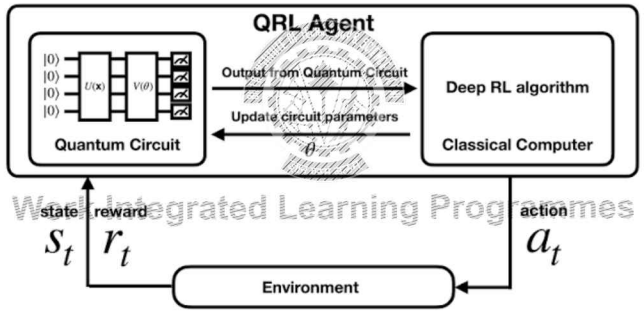
Value-Based Methods	**Policy-Based Methods**
Learn $Q(s, a)$	Learn $\pi(a\|s)$
Action chosen using $\arg\max Q(s, a)$	Policy directly outputs action probabilities
Representative algorithm: DQN	Representative algorithm: REINFORCE
Indirectly learns a policy	Directly learns the policy

Where Does Quantum Computing Enter?

Classical Deep RL	**Quantum RL**
State	State
⇓	⇓
Neural Network	Data Encoding
⇓	⇓
Q-values / Policy	Parameterized Quantum Circuit
⇓	⇓
Loss	Expectation Values
⇓	⇓
Backpropagation	Parameter-Shift Rule

Everything else in the RL algorithm remains unchanged.

Quantum RL

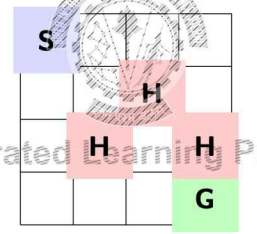


Session Roadmap

- **Part A:** RL foundations - the value-based vs. policy-based split that structures the whole session
- **Part B:** Value-based QRL (Skolik et al.) - PQC replaces the Q-function inside DQN
- **Part C:** Policy-based QRL (Jerbi et al.) - PQC replaces the policy directly, trained via REINFORCE
- **Part D:** Synthesis - comparison and honest assessment vs. classical deep RL
- **Key point:** the PQC itself never changes across Parts B/C - only *what it's trained to output* (a Q-value vs. a policy) changes

A Grid-World Agent

- State s: agent's cell in an $n \times n$ grid; Actions: up/down/left/right
- Reward: $+1$ at goal, -1 in a hole, 0 otherwise



- Same structure as **FrozenLake** - the benchmark used later in Skolik et al.

Episodes, Return and the Learning Goal

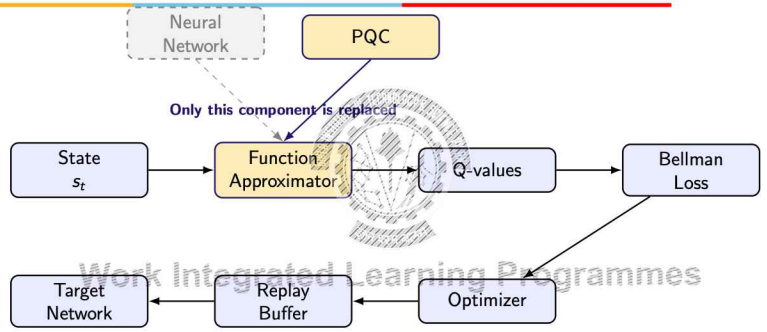
- An **episode**: $s_0, a_0, r_1, s_1, \ldots, s_T$. The **return** from time t:

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad \gamma \in [0, 1)$$

- **Why γ?** Keeps G_t finite; also controls myopic ($\gamma \to 0$) vs. far-sighted ($\gamma \to 1$) behaviour
- **Goal of RL:** maximize $\mathbb{E}[G_0]$ - not just the immediate reward r_1

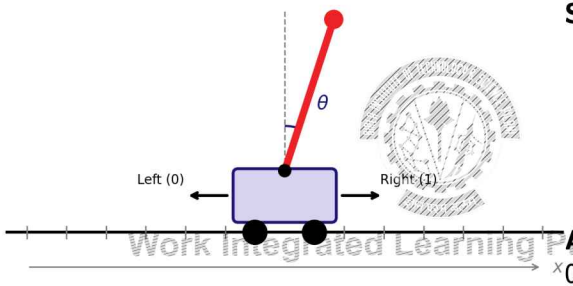
Table of Contents

From Classical DQN to Quantum DQN



- Replay buffer, target network, Bellman loss and optimizer remain exactly the same.
- Only the classical neural network is replaced by a parameterized quantum circuit (PQC).
- The PQC produces Q-values, allowing the rest of the DQN algorithm to operate unchanged.

Running Example: CartPole Control



State $s = (x, \dot{x}, \theta, \dot{\theta})$

- Cart position
- Cart velocity
- Pole angle
- Pole angular velocity

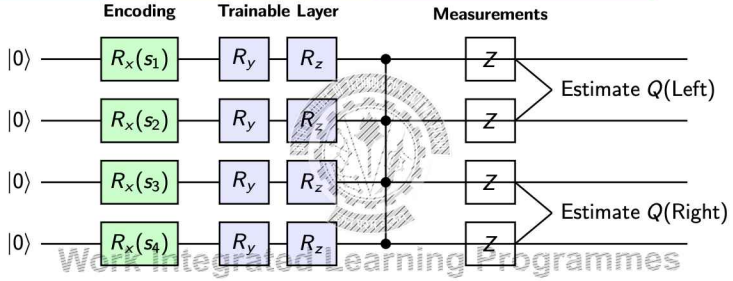
Actions

0 = Move Left

1 = Move Right

Goal: Keep the pole balanced as long as possible.

The PQC Architecture (Skolik et al.)



- **Encoding:** Encode the four CartPole state variables into qubit rotations.
- **Trainable Layer:** Learn the Q-function by optimizing the circuit parameters.
- **Measurements:** Produce one score for each possible action.

From Quantum Measurements to Q-values

Suppose the CartPole agent observes the state

$$s = (x, \dot{x}, \theta, \dot{\theta})$$

The encoded quantum circuit is executed once, but we measure different observables.

Observable Measured	Expectation Value	Interpreted as
O_{Left}	0.82	$Q(\text{Left}) = 0.82$
O_{Right}	0.31	$Q(\text{Right}) = 0.31$

Since $Q(\text{Left}) > Q(\text{Right})$, the DQN agent chooses the **Left** action.

Understanding Expectation Values

Suppose we measure the observable corresponding to the **Left** action four times.

Measurement	1	2	3	4
Outcome	+1	+1	-1	+1

Average (Expectation Value)

$$\frac{(+1) + (+1) + (-1) + (+1)}{4} = 0.5$$

Therefore,

$$Q(\text{Left}) = 0.5$$

Can Every Q-value Be Represented?

Recall our previous example:

Action	Estimated Q-value
Move Left	0.82
Move Right	0.31

Now suppose the agent becomes much more experienced:

$$Q(\text{Left}) = 125, \qquad Q(\text{Right}) = 118$$

But every quantum measurement satisfies $-1 \leq \langle O \rangle \leq 1$

A quantum measurement can never produce 125.

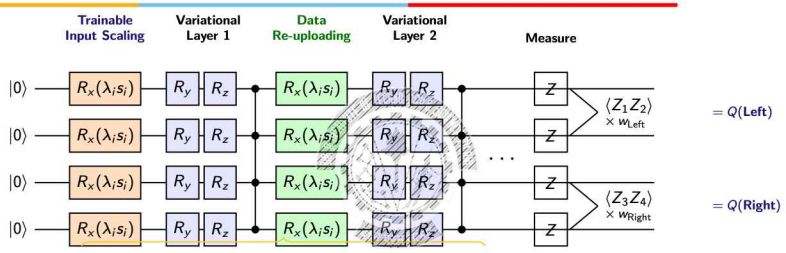
Making the Quantum DQN Practical

The naive Quantum DQN needs a few improvements before it can learn effectively.

Challenge	Modification	Why it helps
Expectation values are bounded	Trainable output scaling	Represent larger Q-values required in RL
Circuit may be too simple	Data re-uploading	Learn richer and more expressive Q-functions
Input features have different ranges	Trainable input scaling	Improve encoding of classical state variables

These modifications improve the **representation** of the Q-function; the underlying DQN algorithm remains unchanged.

Final Quantum DQN Architecture



- The CartPole state is encoded multiple times using data re-uploading
- Trainable parameters learn both the quantum circuit and the feature scaling (λ_i at input, multiplicative w_a at output: $Q(s,a) = w_a \cdot \langle O_a \rangle$)
- Measurements estimate the Q-values used by the classical DQN algorithm

Value-Based QRL: The Governing Equations

Step	Equation
Input scaling	$x_i' = \arctan(x_i \cdot w_{d_i})$
Raw circuit output	$Q(s,a) = \langle 0^{\otimes n} \vert U_\theta(s)^\dagger O_a U_\theta(s) \vert 0^{\otimes n} \rangle$
Output scaling (CartPole)	$Q(s,a) = \dfrac{\langle O_a \rangle + 1}{2} \cdot w_{o_a}$
Action selection	$a_t = \arg\max_a Q(s_t, a)$ (ϵ-greedy)
TD target	$q_{\delta_i} = r_{t+1} + \gamma \cdot \max_a \hat{Q}_{\theta_\delta}(s_{t+1}, a)$
Loss	$\mathcal{L} = \dfrac{1}{\lvert\mathcal{B}\rvert} \sum_{b \in \mathcal{B}} (\mathbf{q}_b - \mathbf{q}_{\delta_b})^2$

θ, w_d, w_o are updated jointly from $\mathcal{L}$ via the parameter-shift rule.

Example (1/2): State → Action

State $s = (x, \dot{x}, \theta, \dot{\theta}) = (0.10,\ 0.20,\ -0.05,\ 0.30)$

Step	Computation	Result
1. Input scaling	$x_i' = \arctan(x_i \cdot w_{d_i})$, $w_d = (0.5, 0.3, 2.0, 1.0)$	$x' \approx (0.05, 0.06, -0.10, 0.29)$
2. Circuit output	Execute $U_\theta(s)$, measure Z_1Z_2, Z_3Z_4	$\langle Z_1Z_2 \rangle = 0.42$, $\langle Z_3Z_4 \rangle = -0.18$
3. Output scaling	$Q(s,a) = \frac{\langle O_a \rangle + 1}{2} \cdot w_{o_a}$, $w_o = 92$	$Q(\mathrm{L}) = 65.32$, $Q(\mathrm{R}) = 37.72$
4. Action selection	$a_t = \arg\max_a Q(s_t, a)$, $\epsilon = 0.05$	Move **Left** (prob. 0.95)

State and weight values are illustrative; equations are exact (eq. 10, 11, 19).

Example (2/2): Reward → Learning Update

Agent takes **Left**, environment responds:

Step	Computation	Result
5. Environment	Reward $r = 1.0$, next state $s' = (0.11, 0.25, -0.02, 0.28)$	Transition stored in replay buffer
6. TD target	$q_\delta = r + \gamma \max_a \hat{Q}_{\theta_\delta}(s', a)$, $\gamma = 0.99$. Target net: $\hat{Q}(s', \mathrm{L}) = 88.0$, $\hat{Q}(s', \mathrm{R}) = 81.0$	$q_\delta = 1.0 + 0.99 \times 88.0 = 88.12$
7. Loss	$\mathbf{q} = (65.32, 37.72)$, $\mathbf{q}_\delta = (88.12, 37.72)$	$\mathcal{L} = 259.92$

Gradients of $\mathcal{L}$ w.r.t. θ, w_d, w_o computed via parameter-shift → one joint update.

Target-network Q-values and reward are illustrative; TD target and loss equations are exact (eq. 8, 9).

Do We Always Need to Learn Q-values?

So far, our Quantum DQN agent has followed the same strategy as the classical DQN:

State	Estimate Q-values	Choose Best Action
s	$Q(\mathrm{Left}), Q(\mathrm{Right})$	$\arg\max_a Q(s, a)$

But DQN is not the only Deep Reinforcement Learning algorithm.

Can we replace the neural network in a **policy-based** algorithm with a parameterized quantum circuit as well?

Let's now explore Quantum Policy Gradient methods.

Table of Contents

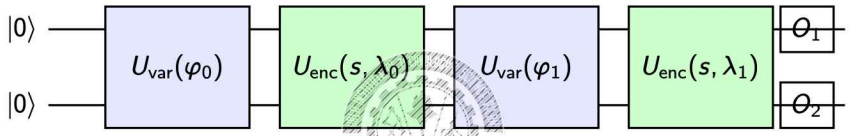
From Q-values to Action Probabilities

Consider the same CartPole state

$$s = (0.20, -0.10, 0.05, 0.30)$$

Value-Based Learning	**Policy-Based Learning**
Estimate	Estimate
$Q(\text{Left}) = 2.8$	$P(\text{Left}) = 0.25$
$Q(\text{Right}) = 1.9$	$P(\text{Right}) = 0.75$
Choose Left	Sample according to 25%, 75%

A Quantum Policy Network



- The policy network is implemented using alternating variational and encoding layers.
- The circuit outputs one score for each possible action, which is converted into action probabilities using Softmax.
- Notice how the overall pipeline is very similar to Quantum DQN - the main difference is how the circuit output is interpreted.

Policy-Based QRL: The Governing Equations

Step	**Equation**
RAW-PQC policy	$\pi_\theta(a\|s) = \langle P_a \rangle_{s,\theta}$
SOFTMAX-PQC policy	$\pi_\theta(a\|s) = \frac{e^{\beta \langle O_a \rangle_{s,\theta}}}{\sum_{a'} e^{\beta \langle O_{a'} \rangle_{s,\theta}}}$
Log-policy gradient	$\nabla_\theta \log \pi_\theta(a\|s) = \beta\left(\nabla_\theta \langle O_a \rangle_{s,\theta} - \sum_{a'} \pi_\theta(a'\|s) \nabla_\theta \langle O_{a'} \rangle_{s,\theta}\right)$
Return (single episode)	$G_t = \sum_{t'=1}^{H-t} \gamma^{t'} r_{t+t'}$
REINFORCE update	$\Delta\theta = \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^{(i)}\|s_t^{(i)}) (G_{i,t} - V_\omega(s_t^{(i)}))$, $\theta \leftarrow \theta + \alpha \Delta\theta$

β is called the **inverse-temperature**: larger $\beta \rightarrow$ more peaked (greedy) policy.

Worked Example (1/2): State → Sampled Action

State $s = (0.20, -0.10, 0.05, 0.30)$, inverse-temperature β=1 (paper's actual CartPole value)

Step	**Computation**	**Result**
1. Circuit output	Execute $U(s, \theta)$, measure O_L, O_R	$\langle O_L \rangle$=0.60, $\langle O_R \rangle$=−0.20
2. Softmax with β	$\pi(a\|s) = \frac{e^{\beta \langle O_a \rangle}}{\sum_{a'} e^{\beta \langle O_{a'} \rangle}}$, $e^{0.60}$=1.82, $e^{-0.20}$=0.82	P(L)=0.69, P(R)=0.31
3. Sample action	Sample from (0.69, 0.31)	Action = **Left**

State, raw scores are illustrative

Worked Example (2/2): Reward → Update

Agent takes **Left**, environment responds with reward r=10 (single-step episode, G_0=10)

Step	Computation	Result
4. Parameter-shift gradients	Illustrative: $\partial_i\langle O_L\rangle$=0.35, $\partial_i\langle O_R\rangle$=−0.10	(one representative angle θ_i)
5. Log-policy gradient	$\beta\left(\partial_i\langle O_L\rangle - \sum_{a'}\pi(a')\partial_i\langle O_{a'}\rangle\right)$	1×(0.35 − 0.2105)=0.1395
6. REINFORCE update	$\Delta\theta_i = \nabla\log\pi(L\|s)\cdot(G_0-V_\omega(s))$, $V_\omega(s)$=0 (unfit)	$\Delta\theta_i$ = 0.1395×(10−0)=1.395

Since the gradient is positive, this update increases P(Left) for similar states going forward.

Step-4 gradients are illustrative

Why Was Softmax Necessary?

Recall that the PQC produces bounded expectation values:

$$-1 \le \langle O_a\rangle \le 1$$

Direct Normalization (RAW-PQC)	Softmax Policy (SOFTMAX-PQC)
Uses expectation values directly to form probabilities	Applies a Softmax function to the PQC outputs
Produces only mildly different probabilities	Can produce highly confident action probabilities
Limited by the bounded measurement range	Inverse-temperature β controls how peaked the policy becomes

Jerbi et al. showed that using a Softmax policy significantly improves learning.

End-to-End Quantum Policy Learning

Workflow

1. Observe state
2. Quantum Policy Network
3. Softmax Layer
4. Action Selection
5. Environment
6. Policy Update

What Happens?

- Receive the current state s_t
- Encode the state into the PQC and measure one score for each action
- Convert the scores into action probabilities
- Sample an action according to the learned probabilities
- Execute the action and receive rewards
- Update the PQC parameters using the classical policy-gradient algorithm

Value-Based vs. Policy-Based Quantum RL

Quantum DQN	Quantum Policy Gradient
PQC estimates Q-values	PQC estimates action preferences
Choose action with largest Q-value	Sample action from Softmax probabilities
Bellman loss updates PQC	REINFORCE updates PQC
Deterministic action selection	Stochastic action selection
Suitable for discrete action spaces	Naturally supports stochastic policies
Good when value estimation is straightforward	Useful when exploration is important

Table of Contents

Where Does PQC-Based RL Stand?

- **Shown:** small PQCs (2–6 qubits) match classical NN performance on small RL benchmarks, with far fewer parameters
- **Not shown:** no advantage on any *natural* RL task - only on an artificial, cryptography-based one
- **Scale:** all results are simulated/small-circuit - untested where classical RL genuinely struggles
- Fewer parameters $\neq$ computational advantage - classical models can also be made parameter-efficient

Open Challenges in Practical QRL

- **Barren plateaus:** gradients vanish exponentially with circuit size - compounds RL's already high-variance gradients
- **Sample efficiency:** every interaction needs a fresh, slower, noisier quantum circuit evaluation
- **Hardware:** results are almost entirely noiseless simulations - unverified at scale on real NISQ devices
- **Noise interaction:** how do quantum measurement/shot noise interact with RL's already-noisy reward signal?

Future Research Directions in Quantum Reinforcement Learning

Current research is moving beyond simply replacing neural networks with PQCs.

Research Challenge	Possible Direction
Low sample efficiency	Classical shadows and improved measurement strategies
Barren plateaus	Problem-inspired ansätze and better initialization methods
Noisy quantum hardware	Error mitigation and noise-aware training
Limited practical advantage	Benchmarking on larger, realistic RL environments
Scalability	Fault-tolerant quantum computers and hybrid architectures

Questions and Takeaways

- Reinforcement learning algorithms remain largely classical.
- Parameterized quantum circuits replace the classical function approximator.
- Quantum Reinforcement Learning is promising, but practical quantum advantage remains an open research question.

Questions?

Thank you :)